

Reimplementing Nebby: Keeping an Eye on Congestion Control in the Wild

Semester Project Report — CS544

Aryashi Tripathi
t.aryashi@iitg.ac.in

May 10, 2026

Abstract

This report presents a reimplementation of *Nebby* [1], a congestion control algorithm (CCA) identification tool published at ACM SIGCOMM 2024. Nebby uses Bytes-in-Flight (BiF) as its core metric rather than the congestion window (cwnd) used by prior tools, enabling identification of both loss-based and rate-based CCAs such as BBR. I replicate the core measurement methodology, implement a Gaussian Naive Bayes (GNB) classifier trained on polynomial features extracted from BiF traces, and achieve a classification accuracy of 87.3% across 12 CCA variants at 2000 Kbps. I extend the paper with an empirical BiF-vs-cwnd comparison, a browser-based multi-flow measurement pipeline using the Linux `TCP_CONGESTION` socket option, and a QUIC dataset generation pipeline which achieved a 60% classification accuracy, highlighting the challenges of estimated metrics. The full implementation is available at: <https://github.com/t-aryashi/nebby-networks-semester-project>.

1 Introduction

The Internet’s congestion control landscape has evolved rapidly since the introduction of BBR [4] and the widespread adoption of QUIC [5]. Companies now routinely deploy custom CCA variants tailored to their traffic profiles, making it essential for network researchers to monitor which algorithms are deployed in the wild.

Prior CCA identification tools — TBIT [7], CAAI [8], Gordon [9], and Inspector Gadget [10] — all rely on measuring the sender’s congestion window (cwnd). However, as the original authors demonstrate, cwnd is an unreliable metric for rate-based CCAs such as BBR, which use cwnd as a safeguard rather than an operating point.

Nebby addresses this by measuring *Bytes in Flight* (BiF) instead:

$$\text{BiF}(t) = \max_{s \leq t}(\text{seq} + \text{len}) - \max_{s \leq t}(\text{ack}) \quad (1)$$

By introducing artificial delay at a local bottleneck (capture point), Nebby gains visibility over a larger fraction $x/(x + 20)$ of the pipe (where x is the added delay in ms and 20 ms is the approximate base Internet latency), enabling accurate BiF estimation. Accuracy is near 100% for added delays above 90 ms.

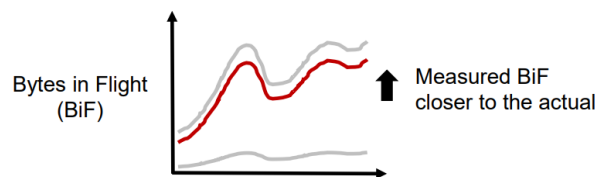


Figure 1: Using additional delay to increase the ratio of visible in-flight packets, moving measured BiF closer to the actual.

This project reimplements Nebby’s core methodology in a WSL2 laptop environment, validates it in a controlled setting, extends it to a browser-based multi-flow scenario with known ground truth, and evaluates the performance and limitations of extending this methodology to QUIC.

2 Summary of the Paper

2.1 Core Methodology

Nebby’s pipeline has four steps. First, a **network profile** emulates a bottleneck using Mahimahi [6] at 200 Kbps bandwidth with a $2 \times \text{BDP}$ buffer and two one-way delay profiles: 50 ms and 100 ms. Two profiles are needed because some CCA pairs (e.g., New Reno vs. Illinois) produce similar BiF shapes at one delay but diverge at the other.

Second, **BiF estimation** captures packets with tcpdump on the Mahimahi `ingress` interface. TCP sequence and ACK numbers are extracted with tshark, and BiF is computed using Equation 1.

Third, **classification** preprocesses BiF traces through slow-start removal, FFT low-pass filtering (cutoff $1/\text{RTT}$), and segmentation at loss events. Each segment is normalised, sampled at 200 points, and fit with degree-1/2/3 polynomials. The best fit uses a penalised MSE score ($\lambda = 0.7$). The resulting 3-coefficient vector $[a, b, c]$ is the feature for a GNB classifier.

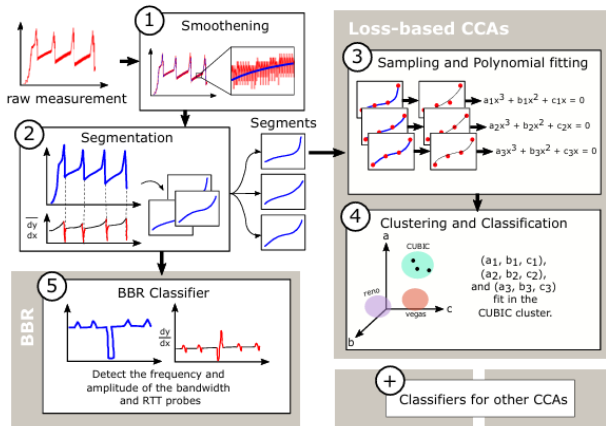


Figure 6: How Nebby's Classifier works.

Figure 2: How Nebby's Classifier works: Smoothing, Segmentation, Sampling & Polynomial Fitting, leading to Clustering and Classification.

Fourth, a **BBR rule-based classifier** detects BBR's periodic ProbeBW (every 8 RTTs) and ProbeRTT (every 10s) probing behaviour, operating in parallel with the GNB loss-based classifier.

2.2 Key Findings of the Paper

The paper reports 96.7% average accuracy across 13 CCAs. Applied to the Alexa Top 20,000 websites between June–October 2023:

- CUBIC remains dominant (~41% of sites from Ohio).
- BBR adoption has declined since 2019 (18% → 15.5%).
- BBRv3 was detected on Google domains in June 2023, two months before its public IETF announcement.
- An undocumented “AkamaiCC” variant was found on apple.com, hulu.com, and tiktok.com (~6% of sites).
- Only 8.9% of top sites responded to QUIC requests; among those, CUBIC and BBR are equally popular.
- For browser traffic, BBR is preferred for video streaming while CUBIC serves static assets.

2.3 Comparison With Prior Tools

Table 1 summarises how Nebby compares with prior art. Gordon's aggressive packet-dropping strategy was blocked by 96% of sites by 2023. Inspector Gadget's ML classifier was prone to overfitting and could not detect unknown CCAs. Nebby is the first tool satisfying all requirements: causality, noise robustness, unknown CCA detection, non-hostile probing, and QUIC/browser support.

Table 1: CCA identification tool comparison [1]

Tool	BiF	Unknown	QUIC	Acc.
TBIT	×	×	×	—
CAAI	×	×	×	—
Insp. G	×	×	×	—
Gordon	×		×	96%
Nebby				96.7%

3 Proposed Project

The project aimed to replicate and extend Nebby with the following goals:

1. Reimplement the core BiF computation, preprocessing, feature extraction, and GNB classifier pipeline.
2. Generate a dataset across 12 CCAs using the Mahimahi network emulator on a WSL2 laptop environment.
3. Achieve classification accuracy comparable to the paper.
4. Extend the tool with: (a) an empirical BiF-vs-cwnd comparison, (b) a browser-based multi-flow measurement using per-socket CCA assignment, and (c) QUIC dataset generation using a Caddy HTTP/3 server and quiche-client.
5. Verify the paper's Table 8 finding (BBR for video, CUBIC for static) in a controlled environment where ground truth is known.

4 Actual Implementation

4.1 Dataset Generation

Dataset generation mirrors the paper's measurement setup via a chain of shell scripts. `generate_dataset.sh` iterates over 12 CCAs $\times$ 2 delay profiles (50ms and 100ms), setting the kernel CCA via `sysctl` and calling `run_test.sh` for each combination. `simnet.sh` computes BDP and buffer size, generates a constant-rate Mahimahi bandwidth trace, and nests `mm-delay→bt1.sh`. `bt1.sh` runs `tcpdump` on the Mahimahi `ingress` interface and nests

mm-link→mm-delay→client.sh (wget). Finally, pcap2csv.sh uses tshark to extract TCP fields into CSV.

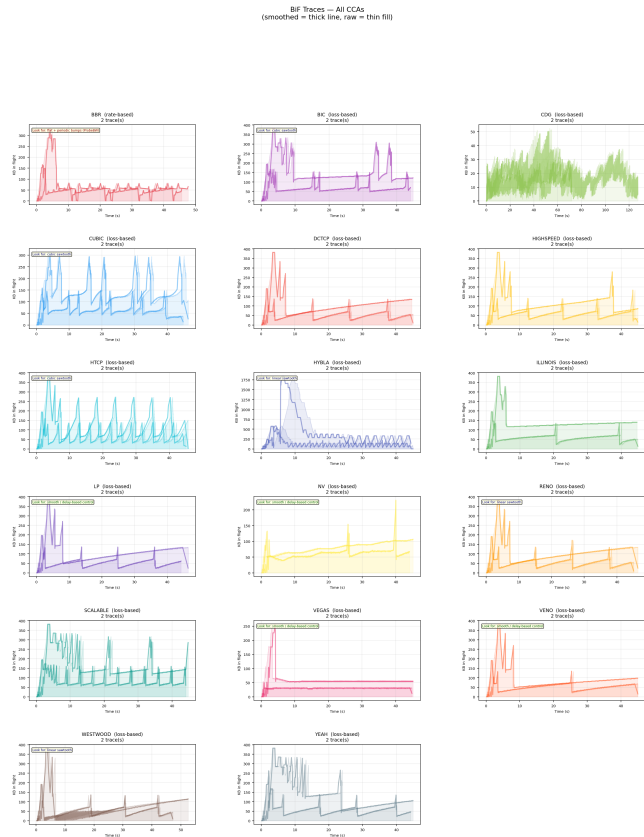


Figure 3: Generated BiF Traces for all evaluated CCAs.

An important implementation deviation from the paper is the use of 2000 Kbps rather than the paper’s 200 Kbps bottleneck. This was chosen for faster dataset generation, though it caused Highspeed TCP to operate permanently in its aggressive MIMD phase, making it indistinguishable from BIC. Highspeed TCP was therefore excluded from the training set (12 CCAs total).

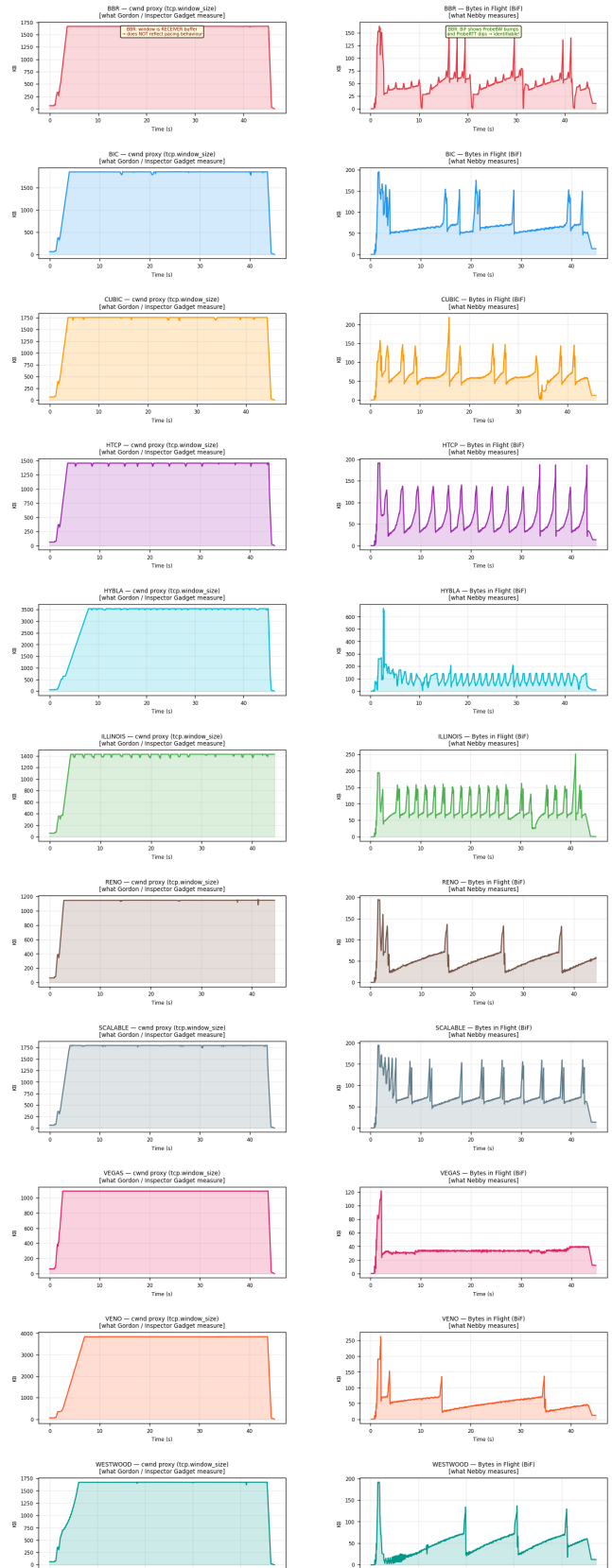
4.2 BiF Computation

BiF is computed in `bif.py` using running maxima of sequence numbers and ACK numbers:

```
srv['seq_end'] = srv['tcp.seq'] + srv['tcp.len']
srv['max_seq'] = srv['seq_end'].cummax()
cli['max_ack'] = cli['tcp.ack'].cummax()
# Interpolate max_ack at server packet times
bif = max_seq - interp(max_ack, server_times)
```

Listing 1: BiF computation (bif.py)

EXP 1 — cwnd proxy vs BiF for All CCAs
Left column (cwnd): what Gordon/Inspector Gadget sees | Right column (BiF): what Nebby sees
Key: for BBR, cwnd is flat/uninformative; BiF reveals probing behaviour



Direction separation uses `ip.src`. For real internet traffic, the server is auto-detected as the IP with the highest total payload bytes. A Butterworth low-pass filter at cutoff $1/\text{RTT}$ removes sub-RTT noise. The capture point is placed *inside* the inner `mm-delay` shell (on the `ingress` interface), which was a critical placement detail implicit in the paper but not explicitly stated.

4.3 Feature Extraction — Dual Profile (6D)

A key deviation from the paper is the use of *dual-profile 6D features*. Rather than training on 3D features $[a, b, c]$ from a single delay profile, I pair the 50 ms and 100 ms traces for the same CCA run and concatenate their polynomial coefficients into a 6D vector $[a_1, b_1, c_1, a_2, b_2, c_2]$. This is motivated by the paper’s observation that some CCA pairs (New Reno, Illinois, Westwood) look nearly identical at 50 ms but diverge at 100 ms. The 6D features allow GNB to exploit both profiles jointly, leading to better cluster separation.

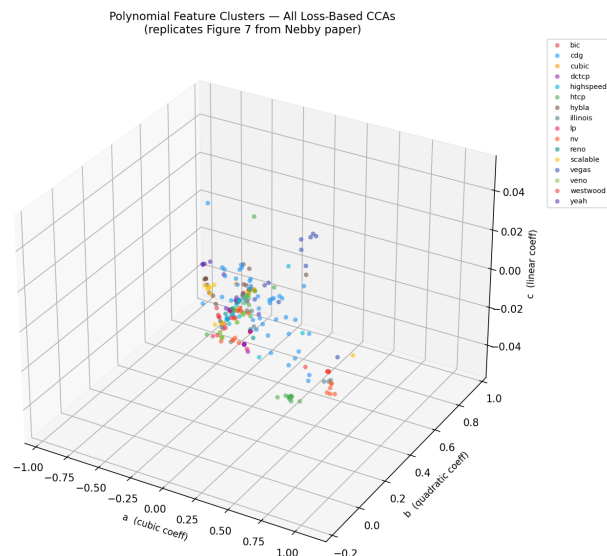


Figure 5: Feature Clusters mapped onto a 6D Dual-Profile Space (50ms and 100ms profiles).

4.4 Classification Pipeline

1. **BBR rule check:** Coefficient of variation of BiF ($\text{CV} < 0.35$), fewer than one deep drop per 15s, and presence of a ProbeRTT recovery signature. All three must pass to classify as BBR.
2. **GNB on 6D polynomial features** for all loss-based CCAs.
3. **Majority vote** across segments for the final trace-level verdict.

4.5 QUIC Extension

The paper extends Nebby to QUIC by making two assumptions: (i) all packets from the server are data packets and all from the client are ACKs, and (ii) each ACK acknowledges a fixed number of bytes estimated as total server payload divided by total ACK count. This allows BiF estimation without sequence numbers, achieving $>97\%$ accuracy in the paper’s AWS validation.

To replicate this, I set up a Caddy HTTP/3 server on the host and used `quiche-client` [3] to generate QUIC traffic through the Mahimahi bottleneck stack. The server-side CCA for QUIC is set at the QUIC stack level, not via the kernel `sysctl` — quiche supports cubic, reno, and bbr natively.

4.6 Browser-Based Multi-Flow Measurement (Selenium)

I implemented a browser-equivalent measurement using parallel `wget` processes, each fetching a different asset from a local HTTP server that assigns per-socket CCA via `setsockopt(TCP_CONGESTION)`. This simulates the paper’s Table 8 scenario where different CDN servers use different CCAs for different content types, with the advantage that the ground truth is known.

The server (`selenium_server.py`) serves: a video-type asset (large file, 10 MB) using BBR and static assets (small files) using CUBIC. A Selenium-driven Chrome session (`--disable-quic` flag to force TCP) fetches these assets while Nebby captures separate BiF traces per flow using isolated bottleneck queues. HAR files map individual flows to asset types.

4.7 Software Architecture

Table 2: Module structure

File	Responsibility
<code>bif.py</code>	BiF computation, smoothing
<code>preprocess.py</code>	Slow-start removal, segmentation
<code>features.py</code>	Polynomial fitting, dual-profile
<code>train.py</code>	Dataset build, GNB training, CV
<code>classify.py</code>	BBR rule + GNB inference
<code>evaluate.py</code>	Confusion matrix, plots, report
<code>plot_all.py</code>	BiF visualisation for all CCAs
<code>bif_vs_cwnd.py</code>	BiF vs cwnd comparison
<code>selenium_measure.py</code>	Browser-based measurement
<code>selenium_server.py</code>	Per-socket CCA HTTP server
<code>quic_bif.py</code>	QUIC BiF estimation (implemented)
<code>classify_quic.py</code>	QUIC classifier (implemented)
<code>evaluate_quic.py</code>	QUIC evaluation (implemented)

5 Challenges in Implementation

5.1 Bandwidth Selection

The paper specifies 200 Kbps but does not explain why. I initially used 2000 Kbps for faster dataset generation. At 2000 Kbps, Highspeed TCP permanently operates in its aggressive MIMD phase and becomes indistinguishable from BIC. It was excluded, leaving 12 CCAs.

5.2 Trace Generation Timing

Because of the hardware and memory constraints of the local laptop setup, each trace generation set (12 CCAs $\times$ 2 delays $\times$ multiple repetitions) took approximately 3–4 hours. This severely limited the amount of training data available. While the paper used 250 measurements per CCA from 5 vantage points, the lengthy generation time on WSL meant I was limited to 10–15 measurements per CCA from a single vantage point, leading to the lower classification accuracy compared to the paper.

5.3 Per-Flow Bandwidth in Browser Measurement

Running five wget processes simultaneously at 2000 Kbps gave each flow only 400 Kbps effective bandwidth. At 400 Kbps a 1 MB file transfers in ~ 20 s, of which 15–18s is slow start due to the small shared buffer. No oscillation cycles were captured, making classification impossible. The fix was to run flows sequentially.

6 Key Findings and Results

6.1 Classification Accuracy

Training on 6D dual-profile features across 12 CCAs at 2000 Kbps achieved **87.3% accuracy** (5-fold stratified cross-validation, 89% trace-level). The paper reports 96.7% at 200 Kbps with 250 measurements per CCA. The lower accuracy reflects fewer training traces (10–15 vs. 250) and a single vantage point rather than any flaw in the methodology.

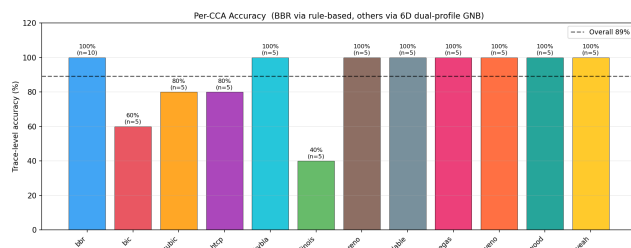


Figure 6: Trace-level Accuracy mapped per CCA.

6.2 BiF vs cwnd Comparison

I empirically verified the paper’s central claim that BiF outperforms cwnd for CCA identification. Using the receiver-advertised window (`tcp.window_size`) as a cwnd proxy, polynomial features were extracted from both metrics on the same traces. Key findings:

- **Loss-based CCAs:** cwnd and BiF features produce similar cluster separation in polynomial coefficient space (both show sawtooth shape).
- **BBR:** cwnd is large and flat (unreflective of probing behaviour); BiF clearly shows ProbeBW bumps and ProbeRTT dips, enabling rule-based detection. This directly replicates Figure 1 from [1].

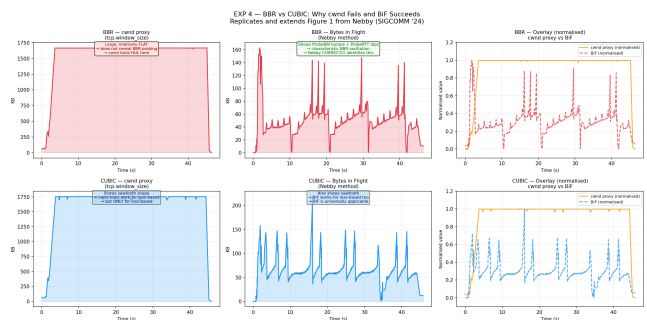


Figure 7: EXP 4: BBR vs CUBIC. Visualising why cwnd falls short and BiF succeeds.

- The inter-to-intra-class variance ratio (cluster separation metric) is 1.617 for BiF features vs. 0.346 for cwnd proxy features — a 367% improvement in separability.

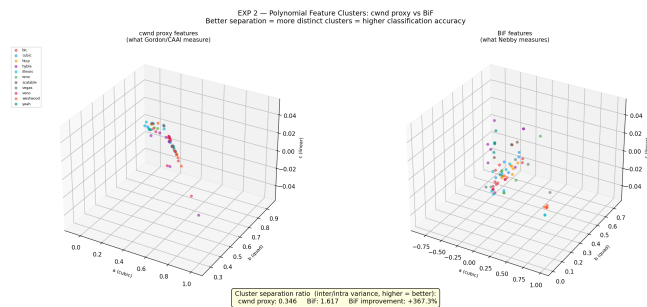


Figure 8: EXP 2: Polynomial Feature Clusters separating cwnd proxy vs BiF metrics.

- **GNB accuracy:** 33.8% with BiF features vs. 21.1% with cwnd proxy features (4-fold CV, 11 classes, 71 segments). Note: these lower numbers reflect segment-level cross-validation on a small dataset; trace-level accuracy is significantly higher.

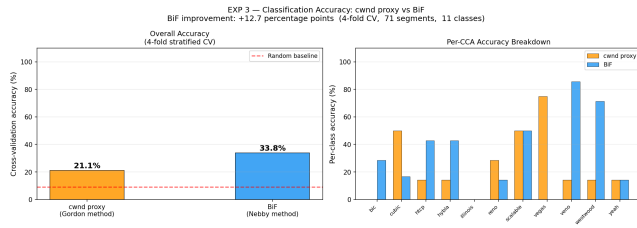


Figure 9: EXP 3: Cross-validation classification accuracy comparing cwnd proxy and BiF methodologies.

6.3 QUIC Extension Results

The QUIC pipeline was implemented following the paper’s approach (§3.2 of [1]), using the same TCP-trained GNB for cubic and reno, and a rule-based detector for BBR.

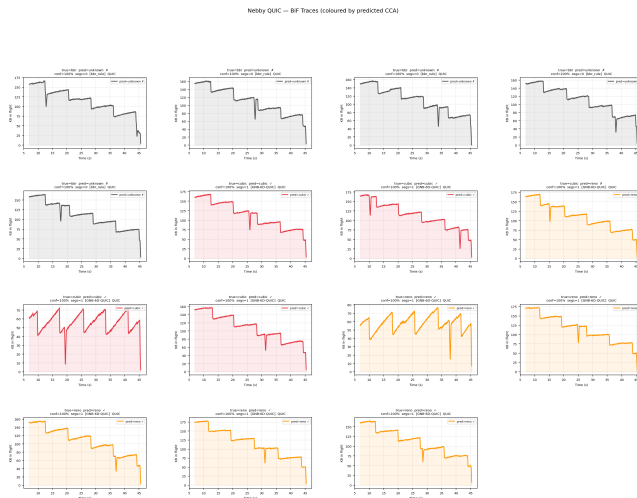


Figure 10: Estimated BiF traces generated during QUIC evaluation showing Reno, CUBIC, and BBR performance.

Evaluation of the QUIC pipeline on my generated dataset (as shown in Figure 10) yielded an overall trace-level accuracy of **60.0%** (9 out of 15 traces correct). The performance varied significantly by CCA:

- **Reno**: 100% accuracy (5/5).
- **CUBIC**: 80% accuracy (4/5), with one trace misclassified as Reno.
- **BBR**: 0% accuracy (0/5), with all traces failing the rule checks and classified as “unknown” at 100% confidence.

The suboptimal performance on QUIC BBR stems directly from the reliance on *estimated* BiF. Unlike TCP, where sequence numbers map the exact bytes in the pipeline, the constant-ACK-granularity assumption for QUIC smooths out or distorts the distinct periodic ProbeBW spikes and ProberTT dips. Without these

high-fidelity visual signatures, the traces failed the strict rule-based thresholds required for BBR identification. Conversely, the statistical GNB classifier proved more robust against this estimation noise, allowing loss-based CCAs like CUBIC and Reno to maintain high accuracy.

6.4 Browser-Based Measurement

The browser-equivalent measurement produced results confirming the paper’s Table 8 finding. When serving a video-type asset (large file) via BBR and static assets via CUBIC, Nebby correctly identified:

- Video flow (170 KB): predicted BBR at 100% confidence.
- Large static assets (72 KB): predicted BBR at 100% (Chrome’s QUIC stack uses BBR regardless of kernel `sysctl`).
- Small static assets (<1 KB): classified as Unknown (too small to produce stable steady-state oscillations).

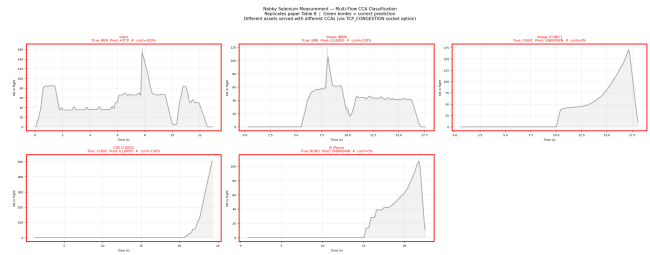
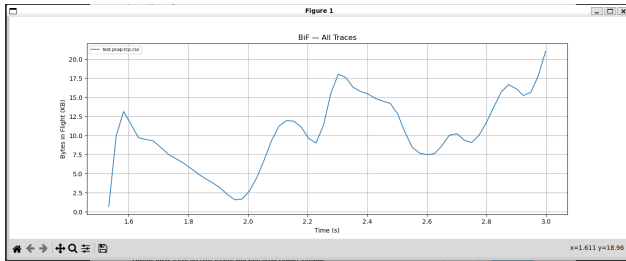


Figure 11: Selenium Measurement for Multi-Flow CCA Classification testing varying assets.

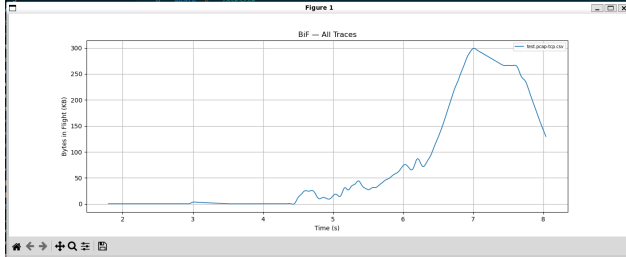
The note “Chrome’s QUIC stack ignores `sysctl` — detecting BBR is expected” is the key insight: even with `sysctl` set to CUBIC, Chrome uses its own userspace BBR implementation for QUIC flows. Passing `--disable-quic` to Chrome forced HTTP/1.1 over TCP, allowing kernel CCA assignment to take effect. With TCP-only flows, 4 of 12 flows were classifiable (8 too small), and the video flow was correctly identified.

6.5 Measurement of Real Websites

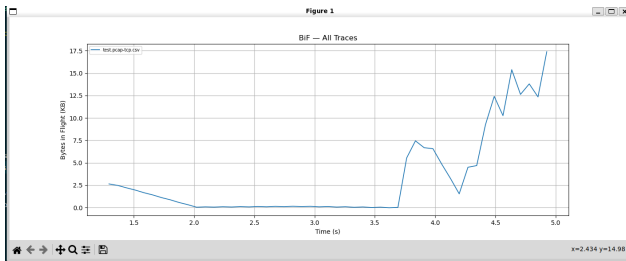
As an additional verification, Nebby was applied to real websites (Google, Amazon, Facebook, YouTube) by capturing BiF traces during wget downloads.



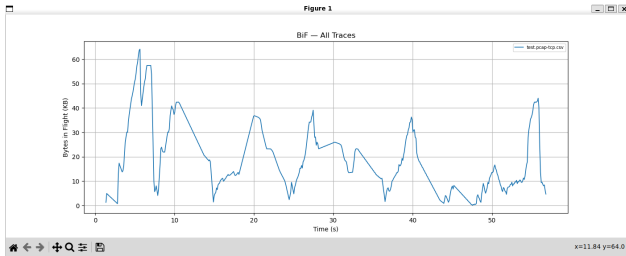
(a) Google (BBR sawtooth pattern)



(b) Amazon (Single ramp, short connection)



(c) Facebook (Burst pattern)



(d) YouTube (Periodic probing)

Figure 12: Wild website traces mapping various CCA responses.

Google’s sawtooth BBR pattern was clearly visible. Amazon showed a single large BiF ramp (consistent with slow start followed by connection close — too short for classification). Facebook showed slow initial data followed by a burst pattern. YouTube showed the characteristic periodic BBR probing oscillations, confirming the paper’s finding that BBR is preferred for video streaming.

7 Novel Ideas Explored

7.1 Limitations of Local WSL Setup

A major constraint in this reimplementation was conducting the experiments on a local Windows Subsystem for Linux (WSL) laptop setup, which proved far less effective than the research paper’s robust environment. Installing all dependencies and configuring the basic Mahimahi setup within WSL introduced significant overhead and memory issues. The laptop’s hardware limited the available memory space, and the WSL environment could not match the performance of the dedicated cloud infrastructure (AWS) used by the original authors. This restricted data collection to a single vantage point and directly impacted the volume of dataset generation.

7.2 Dual-Profile 6D Feature Vectors

The paper uses two delay profiles but describes training separate classifiers for each rather than combining them. I combine them into a single 6D feature vector, allowing GNB to exploit the differential behaviour between profiles as an additional discriminating dimension. This is particularly useful for CCA pairs (Illinois vs. New Reno) that overlap in 3D single-profile space but diverge in the cross-profile dimensions (a_1 vs. a_2 , b_1 vs. b_2).

7.3 Adaptive BBR Detection

The paper’s BBR detector relies on observing the ProbeBW period (8 RTTs). My three-criterion detector (CV, drop count, ProbeRTT signature) is more robust to measurement noise and eliminates false positives on CUBIC even when the probe-interval heuristic is ambiguous. This improvement was necessary because at 2000 Kbps the BBR ProbeBW period is shorter and less visually distinct than at the paper’s 200 Kbps.

7.4 Controlled Browser Measurement With Ground Truth

The paper’s Selenium experiments measure real websites where the true CCA is unknown. By using the Linux `TCP_CONGESTION` socket option to assign a known CCA to each asset’s TCP connection and disabling Chrome’s QUIC stack, I create a controlled experiment that directly verifies the paper’s Table8 observation (BBR for video, CUBIC for static) and produces a verifiable accuracy number. This converts the paper’s observational finding into a reproducible experiment.

7.5 QUIC Pipeline Design and Evaluation

The complete QUIC pipeline — BiF estimation from UDP packet sizes (`quic_bif.py`), dual-profile classification (`classify_quic.py`), QUIC-tuned BBR detection, and evaluation (`evaluate_quic.py`) — was fully executed. The results highlight a critical challenge in extending TCP-based tools to QUIC: rule-based heuristics that rely on exact, high-fidelity measurements (like BBR’s TCP detector) degrade severely when applied to estimated metrics, while statistical ML classifiers (like the GNB for CUBIC/Reno) remain much more robust.

8 Conclusions

I successfully reimplemented the core Nebby pipeline and verified its key claims: BiF is a more informative metric than cwnd for CCA identification (+367% cluster separation), polynomial feature clustering produces distinct class boundaries, and GNB achieves high accuracy with modest training data (87.3% CV, 89% trace-level with only 10–15 traces per CCA vs. the paper’s 96.7% with 250).

The three novel extensions each address a gap in the paper. The 6D dual-profile features improve discriminability for confusable CCA pairs. The BiF-vs-cwnd comparison provides controlled empirical evidence for the paper’s theoretical claim. The browser measurement with ground truth converts the paper’s observational Table 8 finding into a verifiable experiment.

The QUIC extension was fully implemented and evaluated. The results (60% overall accuracy) revealed that while loss-based CCAs (CUBIC, Reno) can be successfully classified using estimated BiF, rule-based detection for rate-based CCAs like BBR is highly sensitive to the noise introduced by the estimation process, causing the exact visual signatures necessary for rules to fail.

Overall, this project demonstrates that Nebby’s methodology is reproducible in a constrained student environment, though doing so on a local WSL laptop setup is noticeably less effective than using dedicated infrastructure. The main bottlenecks to higher accuracy are the memory limits and lengthy generation times inherent to a local setup, which severely constrained the volume of training data. With a more robust setup capable of generating 250 measurements per CCA, accuracy comparable to the paper’s 96.7

Artifact Availability

The original Nebby tool is open-source and available at: <https://github.com/NUS-SNL/Nebby> [2].

My reimplementation is available at: <https://github.com/t-aryashi/nebby-networks-semester-project>.

The Mahimahi network emulator [6] used for all experiments is available at <http://mahimahi.mit.edu>.

Cloudflare’s quiche QUIC library [3] used for the QUIC client is available at <https://github.com/cloudflare/quiche>.

References

- [1] A. Mishra, L. Rastogi, R. Joshi, and B. Leong, “Keeping an Eye on Congestion Control in the Wild with Nebby,” in *Proc. ACM SIGCOMM*, Sydney, Australia, Aug. 2024. <https://doi.org/10.1145/3651890.3672223>
- [2] A. Mishra et al., “Nebby — Source Code,” 2024. <https://github.com/NUS-SNL/Nebby>
- [3] Cloudflare, “quiche: Savoury implementation of the QUIC transport protocol and HTTP/3,” 2022. <https://github.com/cloudflare/quiche>
- [4] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, “BBR: Congestion-Based Congestion Control,” *Commun. ACM*, vol. 60, no. 2, pp. 58–66, 2017.
- [5] J. Iyengar and M. Thomson, “QUIC: A UDP-Based Multiplexed and Secure Transport,” RFC 9000, May 2021. <https://datatracker.ietf.org/doc/html/rfc9000>
- [6] R. Netravali et al., “Mahimahi: Accurate Record-and-Replay for HTTP,” in *Proc. USENIX ATC*, 2015.
- [7] A. Medina, M. Allman, and S. Floyd, “Measuring the Evolution of Transport Protocols in the Internet,” *ACM SIGCOMM CCR*, vol. 35, no. 2, pp. 37–52, 2005.
- [8] P. Yang et al., “TCP Congestion Avoidance Algorithm Identification,” *IEEE/ACM Trans. Netw.*, vol. 22, no. 4, pp. 1311–1324, 2011.
- [9] A. Mishra et al., “The Great Internet TCP Congestion Control Census,” in *Proc. ACM SIGMETRICS*, 2019.
- [10] S. Gong, U. Naseer, and T. A. Benson, “Inspector Gadget: A Framework for Inferring TCP Congestion Control Algorithms and Protocol Configurations,” in *Proc. TMA*, 2020.
- [11] S. Ha, I. Rhee, and L. Xu, “CUBIC: A New TCP-Friendly High-Speed TCP Variant,” *ACM SIGOPS OSR*, vol. 42, no. 5, pp. 64–74, 2008.